

Document Intelligence

Research-style review of layout-aware extraction, document routing, and structured output validation.

Sai Veda

Independent Project Research

2025

Contents

1. Portfolio Abstract	4
2. Document Intelligence Context	5
2.1 Layout-Aware Extraction Problem	5
2.2 Document Review Questions	5
3. Layout And Extraction Foundations	7
3.1 Layout And Extraction Foundations	7
3.2 Observed Document Signals	7
4. Pipeline Architecture And Recovery Flow	8
4.1 Implementation Review Process	8
5. Evaluation Setup And Evidence Collection	9
5.1 Repository Evidence Base	9
5.2 Evaluation Protocol	9
6. Extraction Findings	10
6.1 Benchmark Summary	10
6.2 Interpretation	10
7. Document Evidence Figures	11
7.1 Detected fields on a rendered document (the product view)	11
7.2 Per-field precision / recall / F1	11
8. Validation Review	13
8.1 Document-type router: confusion matrix + routed mix	13
8.2 Extraction KPIs + throughput scaling	13
8.3 Validation Checklist	14
9. Extraction Discussion	15
9.1 Current Limitations	15
10. Review Close	16

10.1 Next Development Stage	16
11. Repository Appendix	17
11.1 Repository Evidence Inventory	17
11.2 Internal References	17

1. Portfolio Abstract

This project documents a layout-aware document AI stack that treats geometry as a primary signal, enabling structured field extraction, table recovery, routing, and search over heterogeneous business documents. The system shows that strong field extraction and routing can be demonstrated credibly without a full OCR or vision stack, as long as the token-plus-box representation and reconstruction logic are engineered carefully. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: Layout-aware field + table extraction, implementation evidence, benchmark results, project validation

2. Document Intelligence Context

Document understanding fails when text is stripped from layout. Invoices, receipts, and forms depend on where a token sits on the page, not only on the token itself, so the pipeline was designed around spatial context from the start. Layout-aware document understanding for invoices, receipts, purchase orders and forms - at scale. Turn a page of tokens-with-boxes into structured, typed fields: identifiers, dates, parties, totals, and the full line-item table - then route the document and make its content searchable. Fully offline, deterministic, no GPU, no paid APIs. Measured on a held-out set of 2,000 documents / 98,687 tokens (train: 8,000 docs / 396,586 tokens), seed-fixed and reproducible with make run: Corpus actually generated: 100,000 documents = 4,962,503 tokens streamed to a 18.8 MB zstd Parquet token table, queried out-of-core with DuckDB.

2.1 Layout-Aware Extraction Problem

The central project problem can be stated directly. Document understanding fails when text is stripped from layout. Invoices, receipts, and forms depend on where a token sits on the page, not only on the token itself, so the pipeline was designed around spatial context from the start. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Document Review Questions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: field macro-F1 0.976; 100k docs, doc-type 100%.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 18 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Layout And Extraction Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. The pipeline stays spatial from the first token box to the recovered fields and tables, which is why the outputs stay structured.

3.1 Layout And Extraction Foundations

- 1 Spatial context is treated as load-bearing because field meaning depends heavily on page position. The pipeline stays spatial from the first token box to the recovered fields and tables, which is why the outputs stay structured.
- 2 Synthetic rendered layouts keep labels exact while still reflecting realistic document structure.
- 3 The pipeline is built for structured outputs, not just page-level classification scores.

3.2 Observed Document Signals

Enterprise document AI (accounts-payable, KYC, claims) has to turn a page image into structured, typed fields: which tokens are the invoice number, the date, the vendor, the totals, and the line-item table - then route the document and make its content searchable. The hard part is that the signal is spatial: "\$1,240.00" is a TOTAL in the bottom-right totals block and an LI_AMOUNT inside the table. A layout-blind text model cannot tell them apart.

- Rendered-layout document generation with gold token boxes.
- Layout-aware feature pipeline and token classification.
- Key-value pairing, table extraction, routing, and search hooks.

4. Pipeline Architecture And Recovery Flow

The pipeline stays spatial from the first token box to the recovered fields and tables, which is why the outputs stay structured. There is no offline OCR/vision stack here, so we generate documents as structured layouts: every document is a list of word-level tokens, each with (text, x, y, w, h) and a gold label. This is a standard, legitimate way to prototype document AI (FUNSD/CORD/DocILE all expose exactly this token+box+label schema) and it buys two things a scanned corpus cannot:

4.1 Implementation Review Process

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Layout modeling: Generated structured pages with gold token positions so every downstream metric could be trusted. Field extraction: Built token-level classification that respects geometry rather than flattening the page to plain text. Structure recovery: Grouped fields into key-value pairs and line-item tables using spatial logic. Document review: Validated extraction quality, routing, and search-readiness on held-out document sets.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Evaluation Setup And Evidence Collection

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 18 automated tests, 1 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 1 benchmark table(s), 4 screenshot asset(s), and 18 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Preserve token geometry all the way through field prediction. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes `benchmarks/results.csv`, `benchmarks/results.md`.

6. Extraction Findings

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: field macro-F1 0.976; 100k docs, doc-type 100%. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The system shows that strong field extraction and routing can be demonstrated credibly without a full OCR or vision stack, as long as the token-plus-box representation and reconstruction logic are engineered carefully. Field-level macro-F1 is strong because the model has access to both text and layout information. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics.

Docs	Tokens	Gen docs/s	Gen tokens/s	Infer docs/s	Infer tokens/s
2,000	99,071	728	36,065	1,838	90,840
10,000	497,136	1,027	51,037	2,262	111,818
50,000	2,482,852	774	38,452	2,108	104,170
100,000	4,964,026	1,009	50,080	2,405	118,876

6.2 Interpretation

Field-level macro-F1 is strong because the model has access to both text and layout information. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics. Routing quality matters operationally because extraction and downstream indexing share the same document understanding layer.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Document Evidence Figures

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Detected fields on a rendered document (the product view)

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. Field-level macro-F1 is strong because the model has access to both text and layout information.

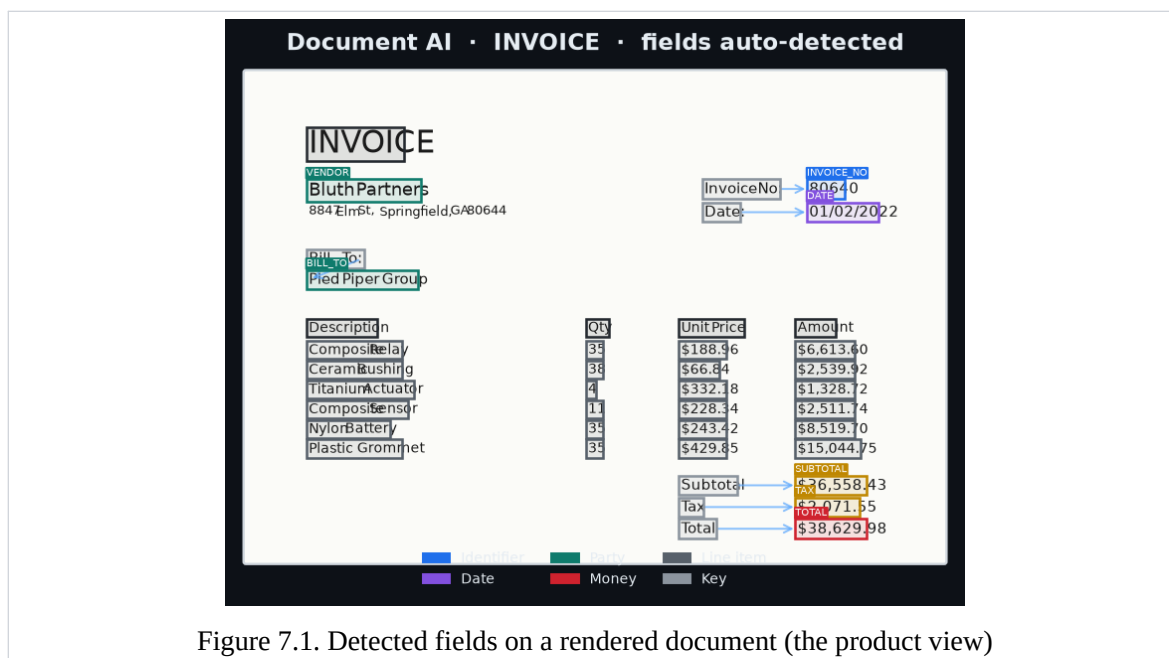


Figure 7.1. Detected fields on a rendered document (the product view)

7.2 Per-field precision / recall / F1

The visual evidence attached to Per-field precision / recall / F1 helps connect the quantitative story to the implemented workflow. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics.

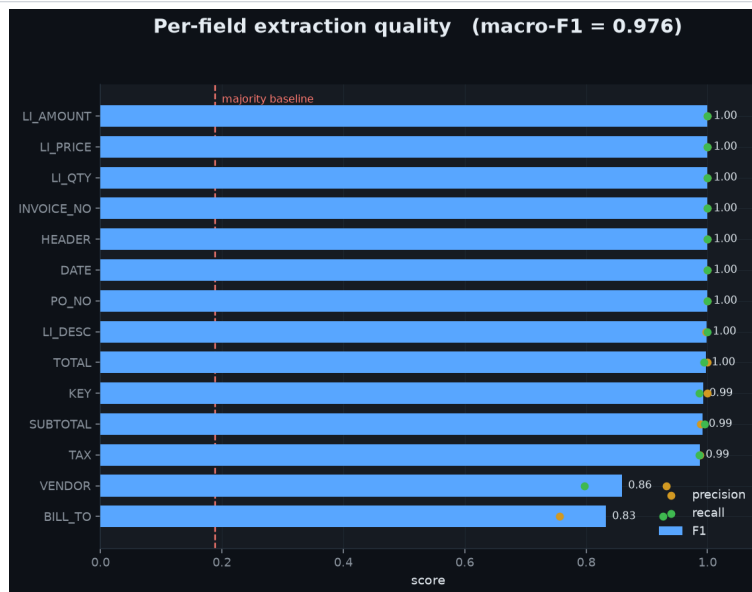


Figure 7.2. Per-field precision / recall / F1

8. Validation Review

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

8.1 Document-type router: confusion matrix + routed mix

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. Routing quality matters operationally because extraction and downstream indexing share the same document understanding layer.

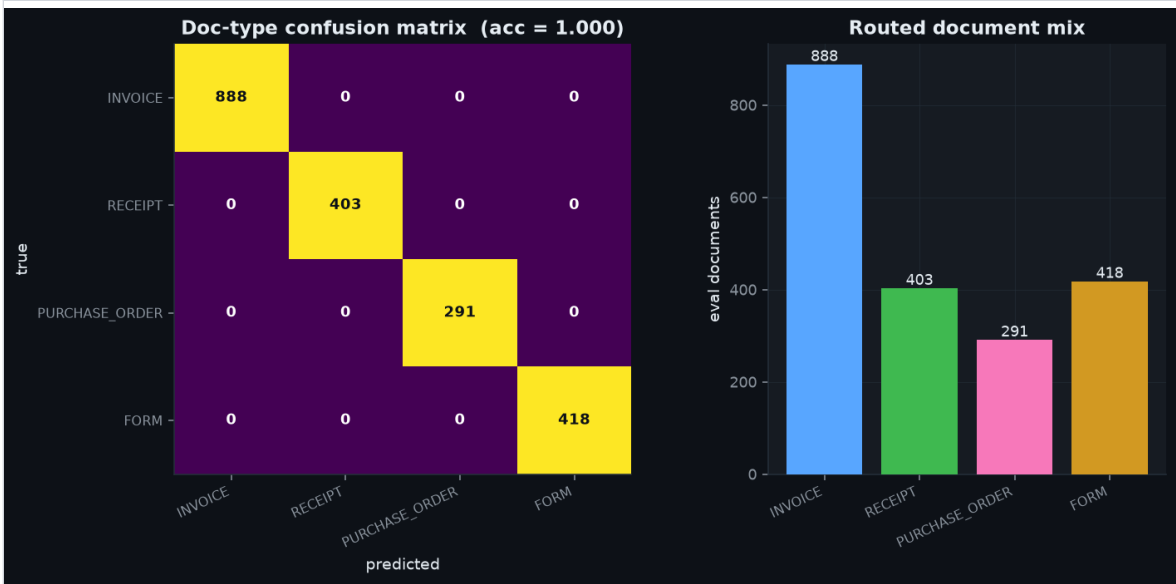


Figure 8.1. Document-type router: confusion matrix + routed mix

8.2 Extraction KPIs + throughput scaling

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Reduced by keeping geometry explicit and by separating field recovery from table reconstruction.

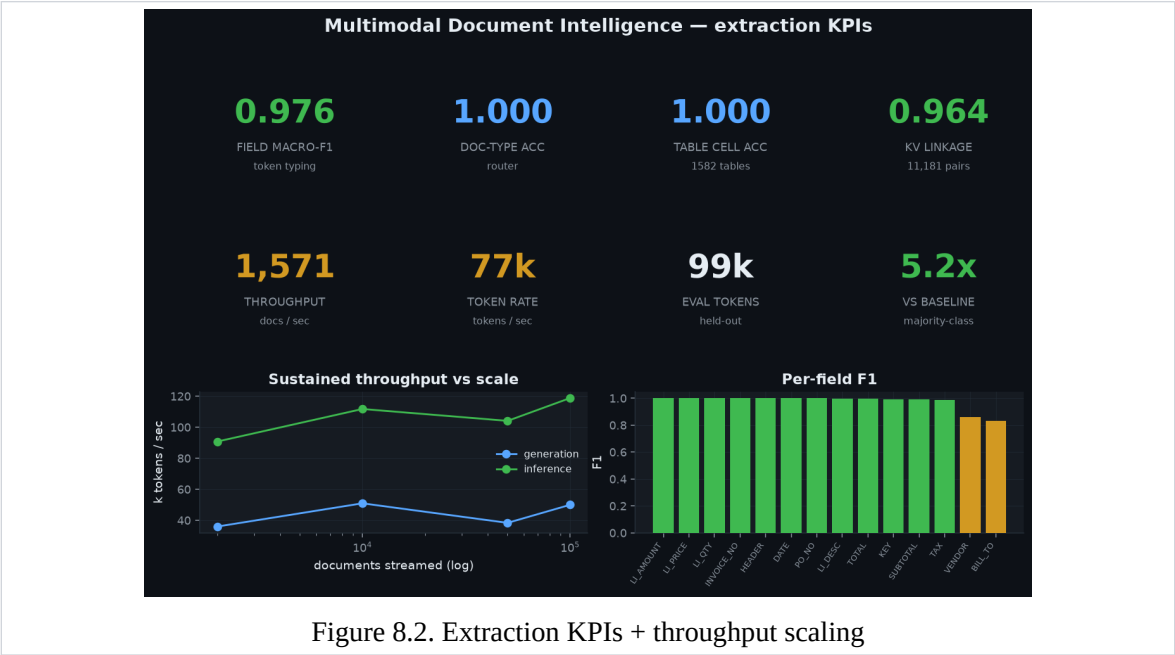


Figure 8.2. Extraction KPIs + throughput scaling

8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	18 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Extraction Discussion

The system shows that strong field extraction and routing can be demonstrated credibly without a full OCR or vision stack, as long as the token-plus-box representation and reconstruction logic are engineered carefully. Field-level macro-F1 is strong because the model has access to both text and layout information. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics. Routing quality matters operationally because extraction and downstream indexing share the same document understanding layer. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Current Limitations

Layout edge cases: Reduced by keeping geometry explicit and by separating field recovery from table reconstruction. OCR mismatch in production: Left visible as a future integration step instead of being hidden inside the prototype. Document-type drift: Addressed by routing first and preserving a modular extraction pipeline.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Review Close

This project document concludes that Document Intelligence is best understood as a complete technical implementation supported by repository-native evidence. This project documents a layout-aware document AI stack that treats geometry as a primary signal, enabling structured field extraction, table recovery, routing, and search over heterogeneous business documents.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Next Development Stage

- Add scanned-document or OCR-noise simulation to stress the layout pipeline further.
- Support more document families and more complex nested table structures.
- Connect extracted entities to downstream review and retrieval workflows.

11. Repository Appendix

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/results.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/results.md	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/detected_document.png	Screenshot	Visual validation surface referenced as 'Detected fields on a rendered document (the product view)' in the document review.
assets/field_f1.png	Screenshot	Visual validation surface referenced as 'Per-field precision / recall / F1' in the document review.
assets/doc_type.png	Screenshot	Visual validation surface referenced as 'Document-type router: confusion matrix + routed mix' in the document review.
assets/dashboard.png	Screenshot	Visual validation surface referenced as 'Extraction KPIs + throughput scaling' in the document review.
tests/	Validation	Automated test suite containing 18 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/results.md - benchmark or result artefact used in the analysis section.
5. tests/ - automated validation suite used to support implementation claims.
6. assets/dashboard.png - generated screenshot evidence included in the report.

7. assets/detected_document.png - generated screenshot evidence included in the report.